Smart Fridge Chef – Proof of Concept

Rozpoznawanie składników przez Vision LLM i przepisy z bazy (RAG)

Kajtek · 29.10.2026 · Status: propozycja do decyzji zespołu

W skrócie

Proponuję, żebyśmy zaczęli od prostszej wersji niż w architekturze Igora: **dwa modele językowe zamiast kaskady YOLO i Vision LLM**. Pierwszy (Vision LLM) rozpoznaje składniki i ich ilości ze zdjęcia lodówki. Drugi, tańszy model tekstowy, wybiera i dopasowuje przepisy z gotowej bazy. Własnego YOLO na tym etapie nie trenujemy.

Cel PoC: sprawdzić, czy cały łańcuch *zdjęcie → lista składników → przepis* działa wystarczająco dobrze, i mieć pierwsze liczby (trafność, czas, koszt) na prezentację. O YOLO decydujemy po PoC, kiedy będziemy znali wyniki i termin oddania. Na końcu dokumentu jest lista decyzji do podjęcia.

Co się zmienia względem architektury Igora

Element	Architektura (Igor)	PoC (propozycja)
Rozpoznawanie	YOLO s/m, Vision LLM tylko dla niepewnych wycinków	Tylko Vision LLM, całe zdjęcie
Trening modelu	Fine-tuning YOLO na RTX 5060, wersje v1–v3	Brak. Zamiast tego porównanie kilku Vision LLM
Podgląd na żywo	YOLO nano w przeglądarce, ramki na żywo	Brak. Zwykłe zdjęcie i przycisk „Skanuj"
Dane do przygotowania	Kilkaset zdjęć lodówek z ręcznie zaznaczonymi ramkami	30–50 zdjęć z listą „co tu jest i ile", bez ramek
Przepisy	Baza + LLM (RAG)	Bez zmian, mniejsza baza na start
Walidacja alergenów i kalorii	W kodzie, nie w LLM	Bez zmian
Słownik składników	Wspólny fundament całego systemu	Bez zmian, wersja startowa 50–100 składników

Wszystko poza rozpoznawaniem zostaje tak, jak zaprojektował Igor. PoC to w praktyce jego etap 1 („Szkielet") z dodaną ewaluacją.

Dlaczego bez YOLO na start

Sam trening w Ultralytics to kilka linijek kodu. Pracochłonne jest wszystko dookoła: zebranie kilkuset zdjęć prawdziwych lodówek, ręczne zaznaczenie ramki wokół każdego produktu, konfiguracja RTX 50 (wymaga PyTorcha z CUDA 12.8 lub nowszą) i dobranie progów kaskady. To kilka tygodni pracy jednej osoby, zanim cokolwiek zadziała end-to-end. Gotowych zbiorów z lodówkami jest mało i są małe: na Roboflow Universe po około 1000 zdjęć, każdy z innymi klasami.

Przypadek	Jak radzi sobie YOLO
Jajka, owoce, warzywa, butelki (wyraźne kształty)	Dobrze i szybko, dobrze liczy sztuki
Różne ułożenie produktu (pionowo, poziomo)	Dobrze, augmentacje przy treningu to załatwiają

Przypadek	Jak radzi sobie YOLO
Mleko, kefir, maślanka w podobnych kartonach	Nie rozróżni. Różnica jest w napisie, a YOLO nie czyta tekstu
Ten sam produkt od wielu producentów	Słabo, potrzeba dużo przykładów każdej marki
Szynka w papierze, resztki w pojemniku	Słabo

Vision LLM czyta etykiety i rozpoznaje produkty spoza listy klas, czyli robi dokładnie to, z czym YOLO ma problem. Słabiej liczy sztuki i potrafi dać inną odpowiedź przy kolejnym wywołaniu. Łagodzą to ekran korekty i ustawienia opisane niżej.

Do tego wykład 4 jest w całości o LLM-ach (RAG, embeddingi, prompty, halucynacje, niedeterminizm). Dobrze zrobiony pipeline z dwoma LLM-ami i RAG pasuje do przedmiotu. Trening YOLO był naszym wyborem, a nie wymogiem prowadzącego.

Jak działa PoC



Na niebiesko: dwa wywołania LLM. Reszta to zwykły kod w Pythonie.

1. **Zdjęcie.** Użytkownik robi zdjęcie lodówki albo kilka, półka po półce. Backend zmniejsza je przed wysłaniem, bo koszt zależy od rozdzielczości.
2. **Vision LLM.** Dostaje zdjęcie i listę składników ze słownika. Zwraca JSON: ID składnika, ilość, jednostka, pewność. Produkt spoza słownika oznacza jako „inny" z krótkim opisem. Temperatura 0 i stały prompt, żeby wyniki były powtarzalne.
3. **Ekran korekty.** Użytkownik zmienia ilości, usuwa i dodaje pozycje. Poprawki zapisujemy, bo to później gotowe dane do ewaluacji albo treningu.
4. **Wyszukiwanie kandydatów (bez LLM).** Filtry SQL (alergeny, dieta, wykluczone składniki), potem ranking według odsetka składników przepisu, które są w lodówce. Wynik: około 20 przepisów.
5. **LLM tekstowy (tańszy model).** Wybiera 3 przepisy, przelicza porcje, proponuje zamienniki, wypisuje brakujące składniki i przyprawy podstawowe. Zwraca JSON według schematu w Pydantic.
6. **Walidacja w kodzie.** Każdy składnik musi istnieć w słowniku, żaden nie może zawierać alergenu użytkownika, kalorie liczy backend. Przepis, który nie przejdzie, odpada.

Gdy żaden przepis z bazy nie pasuje, LLM generuje przepis od zera. Jest oznaczony jako „wygenerowany przez AI" i przechodzi tę samą walidację.

Zakres PoC

W PoC	Później (poza PoC)
Słownik 50–100 składników: nazwy PL i EN, alergeny, kalorie	Pełny słownik z DE i trwałością produktów
Rozpoznawanie przez Vision LLM i porównanie 3–4 modeli	YOLO, kaskada, podgląd na żywo
Baza kilku tysięcy przepisów z jednego źródła	Pełny import, embeddingi, wyszukiwanie hybrydowe
Dobór przepisów przez LLM i walidacja w kodzie	Tłumaczenia, personalizacja z ocen przepisów
Najprostszy interfejs: zdjęcie → lista → przepisy	PWA, konta, stan lodówki, terminy, powiadomienia
Skrypt ewaluacji na zbiorze testowym	

Interfejs w PoC może być najprostszy możliwy (np. Gradio albo prosta strona), bo sprawdzamy pipeline AI. Osoba od frontendu może równolegle zacząć szkielet PWA z kamerą na tym samym API, ale PoC od tego nie zależy.

Jak sprawdzimy, czy działa

Zbiór testowy: 30–50 zdjęć naszych lodówek i lodówek rodzin, w różnym świetle i przy różnym zapełnieniu. Do każdego zdjęcia ręcznie spisujemy listę „co tu jest i ile". Nie rysujemy ramek, więc to kilka wieczorów pracy, a nie kilka tygodni.

Metryka	Co mierzy	Proponowany próg (do ustalenia)
Precision, recall, F1 listy składników	Czy lista ze zdjęcia jest dobra	F1 ≥ 0,8 dla typowych produktów
Błąd ilości	Czy ilości się zgadzają	Ustalimy po pierwszych wynikach
Powtarzalność	To samo zdjęcie 5 razy: czy lista jest ta sama	≥ 90% zgodnych pozycji
Czas skanu	Ile czeka użytkownik	< 10 s
Koszt skanu	Ile płacimy za jedno zdjęcie	Zmierzony i zapisany dla każdego modelu
Przepisy	Odsetek po walidacji, nasza ocena 1–5	0 przepisów z alergenem użytkownika

Porównanie 3–4 Vision LLM (np. Claude, GPT, Gemini, w wersji tańszej i mocniejszej) na tym samym zbiorze daje jedną tabelę: trafność, czas i koszt. Zastępuje porównanie „YOLO / VLM / hybryda" z architektury Igora i od razu odpowiada na otwarte pytanie, którego dostawcy użyć.

Plan i podział pracy

Krok	Kto	Rezultat
1. Słownik startowy 50–100 składników	Wszyscy razem	Tabela składników
2. Zdjęcia testowe i spisanie list	Wszyscy (każdy swoją lodówkę i lodówki rodzin)	30–50 zdjęć z listami
3. Prompt Vision LLM, schemat JSON, skrypt ewaluacji, porównanie modeli	Kajtek	Tabela wyników
4. Import podzbioru przepisów, normalizacja do słownika, wyszukiwanie SQL	Osoba 2 (backend)	Baza przepisów i endpoint
5. LLM do przepisów i walidacja alergenów oraz kalorii	Osoba 2 (backend)	Endpoint zwracający 3 przepisy
6. Prosty interfejs end-to-end	Osoba 3 (frontend)	Demo: zdjęcie → przepisy
7. Podsumowanie wyników i decyzja o YOLO	Wszyscy	Plan kolejnego etapu

Kroki 1 i 2 robimy najpierw, bo od słownika i zbioru testowego zależy reszta. Kroki 3–6 mogą iść równolegle. Orientacyjnie to 2–3 tygodnie, ale zależy to od terminu oddania, którego jeszcze nie znamy.

Co z YOLO po PoC

- **Zostajemy przy samym Vision LLM**, jeśli wyniki są dobre, a termin bliski.
- **Dodajemy gotowy detektor bez treningu** (YOLO-World) tylko do liczenia sztuk i ramek. Etykiety dalej czyta LLM.
- **Pełny fine-tuning YOLO i kaskada** z architektury Igora, jeśli zostanie czas albo zechcemy obniżyć koszt API w produkcie. Poprawki użytkowników z ekranu korekty będą wtedy darmowymi danymi.

Uwaga o kosztach: hybryda obniża koszt tylko wtedy, gdy YOLO sam pewnie obsłuży większość produktów. Jeśli do LLM trafia kilka wycinków z jednego zdjęcia, może to kosztować tyle co jedno całe zdjęcie albo więcej. Tańsze sposoby na start: mniejszy model, niższa rozdzielczość, jedno wywołanie na skan i zapisywanie wyników.

Ryzyka

Ryzyko	Co robimy
Zatłoczona lodówka, coś zostaje pominięte	Kilka zdjęć półka po półce, ekran korekty
LLM zmyśla alergeny albo kalorie	Walidacja w kodzie na ID ze słownika, kalorie liczy backend
Niedeterminizm (inna odpowiedź przy kolejnym wywołaniu)	Temperatura 0, stały prompt, schemat JSON, pomiar powtarzalności
Koszty API	Tańszy model, mniejsza rozdzielczość, limit skanów, zapis kosztu każdego skanu
Prowadzący oczekuje Pythona	Cała część AI i backend w Pythonie. Zapytać, czy frontend może być w TypeScript
Licencje danych (np. RecipeNLG tylko do użytku niekomercyjnego)	Import przepisów jako osobny, wymienialny moduł, pole source przy każdym przepisie

Do decyzji zespołu

1. Czy zaczynamy od PoC z dwoma LLM-ami, a o YOLO decydujemy po PoC?
2. Które modele porównujemy w teście? Propozycja: po jednym tańszym i jednym mocniejszym od 2–3 dostawców.
3. Źródło przepisów na PoC: RecipeNLG czy Food.com? Propozycja: Food.com z Kaggle, bo jest mniejszy i ma kalorie. Licencję sprawdzimy przed importem.
4. Kto pyta prowadzącego o termin oddania i o to, czy frontend może być w TypeScript?